Umucyo Bar (BSSBMS): Full System Documentation

Bar Stock, Sales and Business Management System · written in plain PHP functions · version of 8 November 2026

1. Introduction

1.1 What the system is

Umucyo Bar is a web application that runs on your own computer or server. It helps a bar manage its daily business in one place:

- Products, categories and selling units (bottle, can, crate, pack)
- Suppliers and purchases (buying stock)
- Live stock with a full history of every change
- Sales at the till (Point of Sale) with split payments
- Customers who buy on credit, their limits and their debts
- Cash sessions (opening and closing the till)
- Business expenses
- Users with roles, and an audit log of sensitive actions

1.2 Who uses it

Role	Typical person	Main work
owner	the bar owner	everything, plus users and the audit log
manager	the manager	everything except users and the audit log
cashier	the person at the till	POS, sales history, cash, customers
stock_manager	the store keeper	categories, products, suppliers, purchases, stock

1.3 Technology

Part	Technology
Language	PHP 8 (functions only, no classes)
Database	MySQL / MariaDB
Web server	Apache (WAMP, XAMPP or similar)
Front end	HTML, CSS and a little JavaScript (no framework)
Currency	Rwandan franc (RWF), shown as `1,500 RWF`
Timezone	Africa/Kigali (set in `config/app.php`)

2. Installation

2.1 Requirements

- PHP 8.0 or newer with the `pdo_mysql` extension (enabled by default in WAMP)
- MySQL 5.7+ or MariaDB 10.3+
- Apache with `mod_rewrite` enabled and `AllowOverride All` for the project folder

2.2 Steps (WAMP on Windows)

1. Copy the `bssbms` folder to `C:\wamp64\www\`.
2. Open phpMyAdmin. Run `database/schema.sql` (creates the tables), then `database/seed.sql` (creates the roles, sample accounts and sample data).
3. Open `config/database.php` and check the settings. A default WAMP uses user `root` and an empty password.
4. Open `http://localhost/bssbms/public/` in the browser.
5. Sign in with a sample account.

2.3 Sample accounts

The password for all sample accounts is `Admin@123`. **Change every password before real use.**

Username	Role
admin	owner
manager	manager
cashier	cashier
stock	stock_manager

You can sign in with the username or with the email address.

2.4 Configuration files

`config/app.php`

Key	Meaning
debug	`true` shows real errors on the page (use while developing). `false` hides them (use in real life).
timezone	Used for dates and times, for example `Africa/Kigali`.
session_lifetime	Seconds of inactivity before the user is logged out. The default is 1800 (30 minutes).

`config/database.php`

Key	Meaning
host, port	Where MySQL runs, normally `localhost` and `3306`
database	The database name created by `schema.sql`
username, password	The MySQL account
charset	`utf8mb4`

3. Project structure

```
bssbms/
  .htaccess               sends every address to public/
  config/                 app.php and database.php (settings)
  database/               schema.sql (tables) and seed.sql (starting data)
  public/                 the only folder the browser can reach
    index.php             front controller: every request starts here
    assets/css/           design.css (main design), login.css (login page)
    assets/js/            pos.js, products.js, purchases.js, adjust.js,
login.js
    assets/video/         bg_vd.mp4 (login background)
  routes/web.php          the list of pages: address -> function
  app/
    bootstrap.php         loads every PHP file (one line per file)
    core/                 the toolbox: helpers, database, session, auth,
router
    models/               database queries (read and simple writes)
    services/             business rules that change stock or money
    controllers/          one function per page
    views/                the HTML of every page
      layouts/            main.php (after login), auth.php (login page)
      partials/           sidebar, page header, icons, repeated rows
```

3.1 The four layers

Layer	Folder	Job	Example
Controller	`app/controllers`	Receives the request, reads and checks the form, chooses what to show	`product_store()`
Model	`app/models`	Simple database questions	`search_products()`
Service	`app/services`	Business rules that change several tables together, inside a transaction	`create_sale()`
View	`app/views`	HTML only, with a little PHP to print values	`products/index.php`

3.2 Naming rules

Kind	Style	Examples
Controller (a page)	`module_action`	`product_index`, `product_store`, `sale_receipt`
Model and service	`verb_noun`	`search_products`, `find_customer`, `create_sale`, `deposit_cash`
Core toolbox	short and clear	`url`, `e`, `flash`, `render`, `has_role`

Because controllers are `noun_verb` and everything else is `verb_noun`, two functions never get the same name. PHP stops with an error if two functions share a name.

4. How a request travels

```
Browser -> public/index.php -> app/bootstrap.php -> dispatch_request() ->
controller function

|
            view (HTML)  <-  render()  <-  model and service functions  <----
--+
```

Step	File	What happens
1	`.htaccess`	Every address is sent to `public/index.php`.
2	`public/index.php`	Defines `BASE_PATH` and `APP_PATH`, loads everything, sets the timezone and error display, starts the session, loads the route list and calls the router.
3	`app/bootstrap.php`	Loads the files with `require`, in this order: core, models, services, controllers.
4	`app/core/router.php`	Finds the route and runs the checks below.
5	controller function	Reads input, validates, calls models and services, then shows a page with `render()` or sends the user elsewhere with `redirect()`.
6	models and services	Run the SQL through the `db_*` functions.

Step	File	What happens
7	view	The view is placed inside a layout and sent to the browser.

4.1 The router checks, in order

1. Find the route that matches the method (GET or POST) and the address. If none: **404**.
2. If the route needs a login and the user is not logged in: go to `/login`.
3. Re-read the account from the database (`sync_login`). If it is inactive: log out and go to `/login`.
4. If the route lists roles and the user's role is not allowed: **403**. The owner is always allowed.
5. If the request is a POST and the CSRF token is wrong: **419**.
6. Run the controller function.

4.2 The route list

`routes/web.php` returns one array. Each line is `[method, address, function, options]`.

```
['GET',  '/products',       'product_index', ['roles' => $stockRoles]],
['POST', '/products/store', 'product_store', ['roles' => $stockRoles]],
['GET',  '/login',          'auth_show_login', ['auth' => false]],
```

- `'auth' => false` means a public page (only the login).
- `'roles' => [...]` lists the roles allowed (the owner is always allowed). No roles means any logged-in user.

5. The core toolbox (`app/core`)

5.1 `helpers.php`

Function	What it does
`config('app.debug')`	Reads a setting from `config/`. `config('database')` returns the whole file.

Function	What it does
`url('products')`	Builds a full address such as `/bssbms/public/products`.
`asset('css/design.css')`	Address of a file in `public/assets`.
`redirect('products')`	Sends the browser to another page and stops.
`e($text)`	Makes text safe to print in HTML. Use it on every printed value.
`field($row, 'phone')`	The safe value of a form field, to refill forms.
`rwf(1500)`	Formats money: `1,500 RWF`.
`render($view, $data, $layout)`	Shows a view inside a layout.
`json_response($data, $status)`	Answers `fetch()` calls (the POS) with JSON.
`get_text`, `post_text`, `get_int`, `post_int`, `post_raw`	Read the address bar and forms safely. `post_raw` does not trim (used for passwords).
`is_valid_date`, `looks_like_date`	Date checks. `2026-02-30` is not a valid date.
`db_error_message`	Shows the real database error only when debug is on.
`payment_methods()`	The list `cash`, `mobile_money`, `bank`.

5.2 `database.php`

Every SQL query goes through these functions. Values are always passed as `?` placeholders, so SQL injection is not possible.

Function	Use
`db_all($sql, $params)`	Many rows (empty array when none)
`db_one(...)`	One row, or `null`
`db_value(...)`	One value, for example `COUNT(*)`

Function	Use
`db_column(...)`	One column as a flat list
`db_run(...)`	INSERT, UPDATE, DELETE
`db_insert(...)`	INSERT and return the new id
`db_begin()`, `db_commit()`, `db_rollback()`	Transactions

Transaction. A transaction groups changes so they are saved all together or not at all. Every service that moves stock or money uses this pattern:

```
db_begin();
try {
    // ... several changes ...
    db_commit();
} catch (Throwable $e) {
    db_rollback();
    throw $e;
}
```

5.3 `session.php`

Function	What it does
`boot_session()`	Starts the session. Clears it after 1800 seconds of inactivity and shows "Your session expired".
`flash($type, $message)`	Saves a message shown once on the next page. Types: `success`, `danger`, `warning`.
`flash_pull()`	Returns the waiting messages and clears them. The layout prints them.
`csrf_token()`, `csrf_field()`, `csrf_verify()`	CSRF protection. `csrf_field()` goes inside every POST form.

5.4 `auth.php`

Function	What it does
`login_user($username, $password)`	Finds the user by username or email, checks that the account is active, checks the password with `password_verify`, makes a new session id and stores the user in the session.
`sync_login()`	Re-reads status, name and role from the database on every request.
`is_logged_in()`, `current_user()`, `current_user_id()`, `current_role()`	Who is logged in.
`has_role([...])`	True if the user has one of the roles. The owner always passes.
`logout_user()`	Clears the session.

5.5 `router.php`

`dispatch_request()` runs the checks of section 4.1. `find_route()` looks through the route list.

6. Roles and permissions

Permissions are checked in two places: the route list (`routes/web.php`) blocks the page, and the sidebar hides links the user cannot open. The owner passes every role check.

6.1 Page access

Pages	owner	manager	cashier	stock_manager
Dashboard	yes	yes	yes	yes
Categories, Products, Suppliers, Purchases	yes	yes	no	yes
Stock control (movements, adjustments)	yes	yes	no	yes

Pages	owner	manager	cashier	stock_manager
POS, Sales history, Receipts	yes	yes	yes	no
Cash sessions (open, view, close)	yes	yes	yes	no
Customers, Customer statements, Debt payments	yes	yes	yes	no
Cancel a sale	yes	yes	no	no
Cash deposit, Owner withdrawal	yes	yes	no	no
Activate or deactivate a customer	yes	yes	no	no
Expenses	yes	yes	no	no
Users, Audit log	yes	no	no	no

6.2 Rules inside a page

Some rules depend on the role even on a page the user can open:

- **Discounts** in the POS: only manager and owner. A cashier trying to send a discount is refused.
- **Selling above a customer's credit limit** ("Authorize over limit"): only manager and owner.
- **Customer credit limit and status**: only manager and owner can change them. For a cashier the fields are hidden and the old values are kept.
- **Your own account**: nobody can change their own role or status, or deactivate themselves.
- **The last owner**: the system never allows the last active owner to be deactivated or changed to another role.

7. Database

The tables are created by `database/schema.sql`. **Records are never deleted** when they matter for history. Sales, purchases and categories are cancelled or set to inactive, so old numbers stay correct.

7.1 Tables and their key columns

Table	Purpose	Key columns used by the code
roles	The four roles	id, name
users	Staff accounts	role_id, full_name, username, email, phone, password_hash, status, last_login_at
categories	Beverage categories	name, description, status
products	Products and stock	product_code, category_id, name, brand, barcode, base_unit, purchase_price, selling_price, avg_cost, quantity_in_stock, min_stock, expiry_date, status
product_units	Selling units of a product	product_id, unit_name, conversion_factor, selling_price, is_base
suppliers	Who the bar buys from	supplier_code, name, business_name, phone, email, address, status
purchases	One purchase from a supplier	reference_no, supplier_id, purchase_date, total_cost, amount_paid, balance, status (pending, confirmed, cancelled), created_by, confirmed_by, cancelled_by, cancel_reason
purchase_items	Lines of a purchase	purchase_id, product_id, unit_id, quantity, base_quantity, unit_cost, cost_per_base_unit, total_cost
customers	Customers and credit	customer_code, full_name, phone, address, credit_limit, current_debt, status (active, inactive, blocked)

Table	Purpose	Key columns used by the code
sales	One sale	receipt_no, customer_id, cash_session_id, subtotal, discount, total_amount, amount_paid, credit_amount, status (completed, cancelled), sold_by, discount_by, cancel_reason
sale_items	Lines of a sale	sale_id, product_id, unit_id, quantity, base_quantity, unit_price, unit_cost, subtotal, cost_total
payments	How a sale was paid (one row per method)	sale_id, method, amount, reference, received_by
customer_debts	The unpaid part of a credit sale	customer_id, sale_id, original_amount, paid_amount, balance, status (open, partial, paid, cancelled)
debt_payments	Repayments by customers	customer_id, cash_session_id, amount, method, reference, received_by, notes
cash_sessions	One till shift	opened_by, opening_balance, closed_by, expected_closing, counted_closing, variance, status (open, closed), notes
cash_transactions	Every cash movement of a session	cash_session_id, txn_type, direction (in, out), amount, reference_type, reference_id, description
expense_categories	Kinds of expense	name, status
expenses	Business expenses	expense_code, expense_category_id, cash_session_id, description, amount, payment_method, expense_date, recorded_by
stock_movements	History of every stock change	product_id, movement_type, quantity_change, balance_after, unit_cost, reference_type, reference_id, reason, created_by

Table	Purpose	Key columns used by the code
stock_adjustments	Manual stock adjustments	product_id, adjustment_type, direction, base_quantity, reason, adjusted_by
audit_logs	History of sensitive actions	user_id, action, entity_type, entity_id, details, ip_address

7.2 How the tables connect

- `roles` has many `users`.
- `categories` has many `products`. A `product` has many `product_units`.
- `suppliers` have many `purchases`. A `purchase` has many `purchase_items`, each pointing to a product and a unit.
- `customers` have many `sales`. A `sale` has many `sale_items` and many `payments`.
- A credit `sale` creates one `customer_debts` row. Repayments are saved in `debt_payments`.
- `cash_sessions` have many `cash_transactions`. Sales, expenses and debt payments point to the cash session that was open.
- `stock_movements` point to the thing that caused them with `reference_type` and `reference_id`: a purchase, a sale or a stock adjustment.
- `audit_logs` point to a user and optionally to the record that was changed.

7.3 Codes and references

Item	Format	Example
Product	PRD-0000	PRD-0007
Supplier	SUP-0000	SUP-0003
Customer	CUS-0000	CUS-0012
Expense	EXP-0000	EXP-0020
Purchase	PUR-YYYYMMDD-0000	PUR-20261107-0002
Receipt	REC-YYYYMMDD-0000	REC-20261107-0015

A code is the highest number found plus one, padded to four digits. Purchase and receipt numbers restart every day. If two people save at the same second and get the same number, the save is tried again (up to 3 times).

8. Module reference

For every module this section lists the files, how it works, the checks made on input, and where the data goes.

8.1 Login and logout

Part	File
Pages	`controllers/auth.php`: `auth_show_login`, `auth_login`, `auth_logout`
Login logic	`core/auth.php`: `login_user`, `sync_login`, `logout_user`
Query	`models/users.php`: `find_user_by_username`, `touch_last_login`
View and layout	`views/auth/login.php`, `views/layouts/auth.php`
Design and behaviour	`public/assets/css/login.css`, `public/assets/js/login.js`, `public/assets/video/bg_vd.mp4`

Flow.

1. `GET /login` shows the page. A logged-in user is sent to the dashboard instead.
2. The browser script checks that both fields are filled. "Remember me" saves only the username in the browser's local storage. The password is never stored.
3. `POST /login` sends the form. The router checks the CSRF token. `auth_login()` reads the fields. Empty fields give "Enter your username and password".
4. `login_user()` looks for a user whose **username or email** matches (an exact username match wins). The account must be `active`, and `password_verify()` must accept the password. The stored value is a hash, never the real password.

5. On success a new session id is created (this stops session fixation) and the user is saved in the session. `touch_last_login()` saves the time. The audit log gets `login`.
6. On failure the audit log gets `login_failed` with the typed username, and the page shows "Invalid username or password."
7. `POST /logout` writes `logout` in the audit log, clears the session and returns to the login page.

8.2 Dashboard

Files: `controllers/dashboard.php`, `models/dashboard.php` (`get_dashboard_counts`), `models/categories.php` (`get_category_product_counts`), `views/dashboard/index.php`.

Shows four counts (products, suppliers, customers, users), quick-action buttons filtered by role, and the number of products in each category (top six). It only reads data.

8.3 Categories

Files: `controllers/category.php`, `models/categories.php`, `views/categories/index.php`.

- Name is required, at most 80 characters, and unique. Description is optional, at most 255 characters.
- Categories are never deleted. Toggling sets `active` or `inactive`. Inactive categories disappear from the product form.
- Each change is written to the audit log.

8.4 Products

Files: `controllers/product.php`, `models/products.php`, `views/products/index.php`, `views/products/form.php`, `views/partials/product_unit_row.php`, `public/assets/js/products.js`.

Input checks (`product_collect_input`).

Field	Rule
Name	required, at most 120 characters
Category	must exist

Field	Rule
Brand, Barcode	at most 80 and 50 characters. A barcode must be unique.
Base unit	`bottle` or `can`
Purchase price	number from 0 to 100,000,000
Selling price	number above 0, up to 100,000,000
Minimum stock	whole number from 0 to 1,000,000
Expiry date	optional, must be a real date
Extra units	name must be bottle, can, crate or pack; "contains" a whole number from 2 to 1000; price 0 or more (empty means `contains × base selling price`); each unit name only once

Saving. `create_product()` and `update_product()` run in one transaction, so the product and its units are saved together. The base unit (factor 1) is created and kept in step with the product's base unit and selling price. A unit already used in a purchase or sale cannot be removed.

What is never edited here. `quantity_in_stock` and `avg_cost` only change through confirmed purchases, sales, cancellations and stock adjustments.

Audit. Price changes are logged as `price_change`. Creating, updating and toggling are logged too.

8.5 Suppliers

Files: `controllers/supplier.php`, `models/suppliers.php`, `views/suppliers/*`.

- Name required (at most 120). Business name at most 150. Phone 7 to 20 characters (digits, `+`, spaces, dash). Email must be valid. Address at most 255.
- The list shows for each supplier: number of purchases, total purchased and outstanding balance, with cancelled purchases excluded.
- The history page lists the supplier's purchases with total purchased, total paid and outstanding.

8.6 Purchases

Files: `controllers/purchase.php`, `models/purchases.php`, `services/purchases.php`, `views/purchases/*`, `views/partials/purchase_item_row.php`, `public/assets/js/purchases.js`.

Life of a purchase.

Status	How it is reached	Effect on stock
pending	Saved with `create_purchase()`	none
confirmed	`confirm_purchase()` when the goods arrive	stock increases and average cost is updated
cancelled	`cancel_purchase()`	a confirmed purchase is reversed

Input checks. Active supplier. Real date, not in the future. Notes at most 255 characters. Payment method must be cash, mobile_money or bank. Each line needs an active product, a valid unit, a quantity of 1 to 100,000 and a unit cost of 0 to 100,000,000. At least one line is required. The amount paid now cannot be negative or more than the total.

Confirm. For each line the product row is locked. The new quantity and weighted average cost are saved (formula in section 9). The product's purchase price is set to the cost of one base unit. A `purchase` movement is written.

Cancel. A reason of 3 to 255 characters is required. A pending purchase is only marked cancelled. A confirmed purchase removes its quantities from stock with `purchase_cancel` movements. If part of that stock was already sold, the cancellation is refused with a clear message.

Paying the supplier. The amount must be above 0 and not more than what is still owed. A cash payment is written as cash going out of the open cash session (if one is open). The audit log gets `supplier_payment`.

8.7 POS and sales

Files: `controllers/pos.php`, `controllers/sale.php`, `services/sales.php`, `models/sales.php`, `views/pos/index.php`, `views/sales/*`, `public/assets/js/pos.js`.

Opening the POS. `pos_index()` needs an open cash session, otherwise it sends the user to Cash with a warning. It sends the active products (with units, prices, stock and category), the active customers and a one-time token to the page.

Browser side (`pos.js`). Product tiles, category tabs with counts, search and barcode scanning, the cart with steppers, split payments, live totals and warnings. The Complete sale button is disabled while a sale cannot succeed. The page calculates for display only; the server recalculates everything.

Server side. `pos_checkout()` checks the one-time token (this stops the same sale being sent twice), builds the item and payment lists and calls `create_sale()`. It always answers in JSON: `{ok: true, receipt_url}` or `{ok: false, message}`.

`create_sale()` steps (one transaction).

1. A cash session must be open.
2. The cart cannot be empty. A discount cannot be negative, and only a manager or owner may give one.
3. Payments with an amount of 0 are ignored. Each method must be cash, mobile_money or bank.
4. For each line: quantity 1 to 100,000, a valid unit, an active product. The product row is locked, stock must be enough, and the **price is read from the database**. The stock is reduced.
5. Totals are calculated (section 9). Payments cannot exceed the total.
6. If something is unpaid: a customer is required, must be active, must have a credit limit above 0, and the new debt must stay within the limit, unless a manager or owner authorizes it.
7. The sale, its items, stock movements, payments, cash movements (cash payments only) and the debt are saved. The audit log gets `sale_create`, and `discount` or `credit_limit_override` when they apply.

Cancelling a sale (`cancel_sale`, manager and owner). A reason is required. The stock returns with `sale_cancel` movements. The customer's debt is removed, but only if nothing was repaid yet. The cash part is refunded from the open cash session, so one must be open. The sale stays in history with status `cancelled`.

8.8 Cash management

Files: `controllers/cash.php`, `services/cash.php`, `views/cash/*`.

Action	Function	Rules
Open	`open_cash_session`	opening balance 0 or more, only one session open at a time
Record a movement	`record_cash_transaction`	used by sales, debt payments, supplier payments, expenses, refunds
Deposit	`deposit_cash`	amount above 0
Owner withdrawal	`withdraw_cash`	amount above 0, not more than the expected cash, reason of 3 to 255 characters
Close	`close_cash_session`	counted cash 0 or more, a note is required if it differs from expected

Movement types: `sale`, `debt_payment`, `deposit` (cash in) and `supplier_payment`, `expense`, `owner_withdrawal`, `refund` (cash out). Only cash payments touch the drawer. Credit sales, Mobile Money and bank payments do not.

8.9 Customers and debts

Files: `controllers/customer.php`, `controllers/debt.php`, `models/customers.php`, `services/debts.php`, `views/customers/*`.

- Name required (at most 120). Phone and address checked like suppliers. Credit limit from 0 to 100,000,000; 0 means no credit.
- Only manager and owner can set the credit limit and status (active, inactive, blocked). Changes to the limit are logged as `credit_limit_change`.
- `current_debt` is never typed. It changes only through credit sales, sale cancellations and repayments.
- **Statement.** Credit sales are debits and payments are credits, sorted by date. Each row shows a running balance.
- **Repayment (`pay_customer_debt`).** Amount above 0, method cash, mobile_money or bank. A cash repayment needs an open cash session. The amount cannot be more than the customer owes. The oldest debt is paid first (section 9). The audit log gets `debt_payment`.

8.10 Expenses

Files: `controllers/expense.php`, `models/expenses.php`, `services/expenses.php`, `views/expenses/*`.

- Category must exist. Description 3 to 255 characters. Amount above 0 and at most 1,000,000,000. Date must be real and not in the future. Method: cash, mobile_money or bank.
- A cash expense needs an open cash session and enough expected cash. It is saved together with a cash-out movement.
- Owner withdrawals are not expenses. They live in Cash management.

8.11 Stock control

Files: `controllers/stock.php`, `models/stock.php`, `services/stock.php`, `views/stock/*`, `public/assets/js/adjust.js`.

- **Movements** list every change of stock with the change, the balance after and the reference.
- **Movement types:** purchase, sale, sale_cancel, purchase_cancel, damaged, expired, missing, returned, correction.
- **Adjustment types:** damaged, expired, missing (always decrease), returned (always increases), correction (the user chooses).
- `apply_stock_adjustment()` needs a reason of 3 to 255 characters and a whole quantity of 1 to 1,000,000. The product row is locked. Stock can never go below zero. The movement is valued at the product's current average cost. The audit log gets `stock_adjustment`.
- The form lists expired products still in stock. `adjust.js` previews the stock before and after.

8.12 Users

Files: `controllers/user.php`, `models/users.php`, `views/users/index.php`, `views/users/form.php`.

Field	Rule
Full name	2 to 120 characters

Field	Rule
Username	3 to 50 characters: letters, digits, dot, dash, underscore. Unique.
Email	optional, valid and unique
Phone	optional, 7 to 20 characters
Role	must exist
Password	at least 8 characters with a letter and a digit. Required for new users. When editing, empty keeps the old password.

Passwords are saved with `password_hash` (bcrypt). Role changes and password resets are logged. You cannot change your own role or status, and the last active owner cannot be removed.

8.13 Audit log

Files: `controllers/user.php` (`user_audit`), `models/audit.php`, `services/audit.php` (`audit_log`), `views/users/audit.php`.

`audit_log()` writes the user, action, record type, record id, details and IP address. It never stops the app if saving fails. The page filters by action, user and dates and shows the latest 300 entries. Logged actions include login, login_failed, logout, price_change, discount, credit_limit_override, credit_limit_change, sale_create, sale_cancel, purchase_create, purchase_confirm, purchase_cancel, supplier_payment, debt_payment, expense_create, cash_session_open, cash_session_close, cash_deposit, owner_withdrawal, stock_adjustment, role_change, password_reset, and create, update and activate/deactivate actions on records.

9. Formulas used in the system

This section explains every calculation, in the order a bar owner meets them. Each one has a small example.

9.1 Base quantity (units to bottles)

```
base_quantity = quantity × conversion_factor
```

Stock is always counted in the smallest unit (bottles or cans). `conversion_factor` is how many bottles are inside the unit (bottle = 1, a crate of 24 = 24).

Example. Selling 2 crates of 24: `2 × 24 = 48` bottles leave the stock.

9.2 Purchase amounts

```
cost_per_base_unit = unit_cost ÷ conversion_factor      (rounded to 4
decimals)
line_total         = quantity × unit_cost
purchase_total     = sum of all line totals
balance            = total_cost − amount_paid
```

Example. A crate of 24 bottles costs 12,000. One bottle costs `12,000 ÷ 24 = 500`. Buying 3 crates costs `3 × 12,000 = 36,000`. If you pay 10,000 now, the balance owed is 26,000.

9.3 Weighted average cost

When a purchase is **confirmed**:

```
new_quantity = old_quantity + added_quantity
new_avg_cost = ((old_quantity × old_avg_cost) + (added_quantity × new_cost))
÷ new_quantity
```

This balances different buying prices fairly.

Example. You have 10 bottles at an average cost of 500. You receive 20 bottles at 600. `(10 × 500 + 20 × 600) ÷ 30 = 17,000 ÷ 30 = 566.67`.

When a confirmed purchase is **cancelled**, the opposite is done:

```
new_quantity = current_quantity − removed_quantity
new_avg_cost = ((current_quantity × avg_cost) − (removed_quantity ×
purchase_cost)) ÷ new_quantity
```

The result is never below 0. If the new quantity is 0, the average cost stays as it was. Cancelling is refused when the current quantity is smaller than the quantity to remove.

Example. Continuing the example above: `(30 × 566.67 − 20 × 600) ÷ 10 = 500`. The cost returns to what it was.

9.4 Paying a supplier

```
new_paid = amount_paid + payment
balance  = total_cost − new_paid
```

The payment must be above 0 and `new_paid` cannot be more than `total_cost`.

9.5 Sale totals

```
line_total = quantity × unit_price            (the price is read from the
database)
subtotal   = sum of all line totals
total      = subtotal − discount
paid       = sum of all payments
credit     = max(total − paid, 0)             (what the customer will pay
later)
```

Rules: the discount cannot be above the subtotal, the total must be above 0, and `paid` cannot be more than `total`.

Example. 3 Coca-Cola at 700 gives 2,100. A discount of 100 gives a total of 2,000. The customer pays 1,400, so the credit is 600.

9.6 Credit limit check

```
new_debt = current_debt + credit
refused when new_debt > credit_limit
```

A credit limit of 0 means no credit at all. A manager or owner can authorize a sale above the limit (it is written to the audit log as `credit_limit_override`). In the customers list: `available credit = credit limit − current debt`, and the "Over limit" badge appears when the limit is above 0 and the debt is above the limit.

9.7 Cost and profit data

At the moment of a sale the system saves:

```
unit_cost  = the product's average cost at that moment
cost_total = base_quantity × avg_cost
```

So profit can be worked out later even if costs change:

```
profit of a line = line_total − cost_total
```

The data is stored, but the Reports page is not built yet (the menu shows it as "soon").

9.8 Cash session

```
cash_in  = sale + debt_payment + deposit
cash_out = supplier_payment + expense + owner_withdrawal + refund
expected = opening_balance + cash_in − cash_out
variance = counted_cash − expected
```

Only payments made in **cash** are counted. Mobile Money, bank payments and credit sales do not enter the drawer. A variance other than 0 needs a note when closing.

Example. Opening 50,000. Cash sales 120,000 and debt payments 5,000, so cash in is 125,000. Supplier payment 30,000 and an expense of 10,000, so cash out is 40,000. Expected: `50,000 + 125,000 − 40,000 = 135,000`. You count 134,000, so the variance is `−1,000`.

An owner withdrawal cannot be larger than the expected cash. A cash expense cannot be larger than the expected cash.

9.9 Paying a customer's debt

The customer cannot pay more than they owe in total. The money is applied to the **oldest** debts first. For each debt:

```
pay         = min(money_left, debt_balance)
new_paid    = paid_amount + pay
new_balance = balance − pay             (status becomes "paid" at 0,
otherwise "partial")
money_left  = money_left − pay
```

Then: `customer.current_debt = max(0, current_debt − payment)`.

Example. Debt A is 5,000 (older) and debt B is 4,000, so the customer owes 9,000. They pay 6,000. A gets 5,000 and becomes paid. B gets 1,000 and has 3,000 left. The customer's debt is now `9,000 − 6,000 = 3,000`.

9.10 Stock adjustment

```
base_quantity = quantity × conversion_factor
increase:  new_stock = stock + base_quantity
decrease:  new_stock = stock − base_quantity       (refused if base_quantity
> stock)
value at cost = base_quantity × avg_cost
```

Example. Stock is 48 bottles. Removing 2 damaged crates of 24 gives `48 − 48 = 0`. Removing 3 crates is refused, because stock can never go below zero.

9.11 Customer statement balance

```
balance after a row = previous balance + debit − credit
```

A credit sale is a debit (the customer owes more). A repayment is a credit.

9.12 Supplier outstanding

```
outstanding = sum of balance of the supplier's purchases that are not
cancelled
```

9.13 Low stock and expired

- A product is **Low** when `quantity_in_stock ⩽ min_stock`.
- A product is **Expired** when `expiry_date` is before today. The stock page warns when expired products still have stock.

9.14 Codes, rounding and timeout

- **Next code** = highest number found + 1, padded to four digits (PRD-0007 is followed by PRD-0008).
- **Rounding.** Money is rounded with `round(x, 2)`. Comparisons add a tiny allowance (`> total + 0.009`) so computer decimal errors such as `0.1 + 0.2` never cause a false error.
- **Session timeout.** The user is logged out when `now − last_activity > 1800` seconds.

9.15 Small calculations in the POS screen (`pos.js`)

These are for display only; the server repeats all the real calculations.

What	Formula
Cart badge	sum of all line quantities
Line total	`quantity × unit price`
Fill button	`total − amounts typed in the other payment rows`
On credit	`max(total − paid, 0)`
Stock badge	0 = "Out of stock", 5 or less = "N left", more = "N in stock"
Tile colour	`product id % 8` picks one of eight colours, so a product keeps its colour
Tile initials	first letters of the first two words of the product name
Stock check in the cart	`sum of (quantity × factor)` of all lines of a product must not be more than its stock

10. User guide: how to use each page

This section is for the people who use the system every day. It follows the real screens and fields.

10.1 The right order for a new bar

The pages depend on each other, so set them up in this order:

1. **Users**: the owner creates the staff accounts.
2. **Categories**: Beer, Soft drink, Water, Wine.
3. **Products**: add every product with its prices.
4. **Suppliers**: add who you buy from.
5. **Purchases**: record a purchase, then **confirm** it. This puts the first stock in.
6. **Cash Management**: open the cash session.

7. **POS**: sell.
8. **Customers**: register customers who buy on credit (any time before a credit sale).

Why this order? A product needs a category. A purchase needs a supplier and products. A sale needs stock and an open cash session. A credit sale needs a customer.

10.2 Login

Address: `/login`.

1. Type your **username or email**, for example `admin`.
2. Type your **password**. The eye button shows or hides it.
3. Tick **Remember me** if you want the username filled in next time. It remembers only the username, never the password.
4. Press **Sign in**.

Message	Meaning
Enter your username and password	A field is empty.
Invalid username or password	One of them is wrong, or the account is inactive.
Your session expired	You did nothing for 30 minutes. Log in again.

Forgot password? shows a note to contact the system administrator. The owner resets passwords on the Users page.

10.3 The screen around every page

- **Left sidebar (fixed):** the menu. It shows only what your role may open. The highlighted item is the current page. **Log out** is at the bottom.
- **Top right:** your name.
- **Coloured bars** at the top of a page are messages, such as "Saved" or an error.
- **Tabs** on some pages switch between two views, for example Movements and Adjustment records.

10.4 Dashboard

Who: everyone.

- Four tiles show the number of products, suppliers, customers and users. Click a tile to open that page (if your role allows it).
- **Quick actions** are buttons for New sale, New purchase, Stock adjustment and New product. You only see the ones your role can use.
- The **Statistics** list shows how many products are in each category.

10.5 Categories

Who: manager, stock manager, owner. Menu: Categories.

Add a category. Type the **Name** (for example Beer). It is required, at most 80 characters and must be unique. Optionally type a **Description**. Press **Add**.

Edit. Change the name or description in the row and press **Save**.

Activate or deactivate. Use the status button in the row. An inactive category is hidden from the product form. A category is never deleted, so old products stay correct.

Columns: `#`, Name, Description, Products (how many use it), Status, Actions.

Connected to: the **Category** field of the product form and the category tabs in the POS.

10.6 Products

Who: manager, stock manager, owner. Menu: Products.

The list. Search by name, brand, barcode or code. Filter by category, status and **Low stock only**. Columns:

Column	Meaning
Code	like PRD-0004, made automatically
Product	name, with brand and barcode
Category	the product's category
Stock	quantity now. A red **Low** badge shows when stock is at or below the minimum.
Min	the minimum stock
Avg cost	average cost per bottle, calculated by the system
Selling price	the price per base unit

Column	Meaning
Expiry	a red **Expired** badge shows when the date has passed
Status	active or inactive
Actions	Edit, Activate or Deactivate

Add a product. Press **+ New product**, fill the form and press **Create product**.

Field	What to enter	How it works
Product name *	for example Primus 72cl	Required, up to 120 characters.
Category *	choose one	Only active categories appear.
Brand	for example Bralirwa	Optional, searchable.
Barcode	the number under the bar lines	Optional but unique. In the POS, scanning it and pressing Enter adds the product to the cart.
Base unit *	Bottle or Can	The smallest unit. All stock is counted in it.
Status	Active or Inactive	Inactive products cannot be sold or bought.
Purchase price per bottle *	what you usually pay for one	A reference price that pre-fills the purchase form. The real cost is the average cost from confirmed purchases.
Selling price per bottle *	the price you charge for one	Must be above 0. The POS uses it.
Minimum stock *	for example 24	The reorder warning level. At or below it the product shows **Low**.
Expiry date	optional	Used for the Expired badge and the warning on the stock adjustment page.

Other selling units (crate, pack). Press **+ Add unit (crate, pack…)**. Choose the unit, enter how many bottles it contains (2 to 1000) and its price. If the price is empty, the system uses `contains × base selling price`. Example: a crate of 20 with a bottle price of 1,000 is 20,000. Stock is counted in bottles, so buying 5 crates adds 100 bottles and you can still sell single bottles. Each unit name can be used only once per product.

When you edit a product.

- The top shows the **Product ID** and the **Quantity in stock**. The quantity cannot be typed here. It changes only through Purchases, Sales and Stock adjustments.
- Tick **Remove** to delete a unit. A unit already used in a purchase or sale cannot be removed.
- Price changes are written to the audit log.

Connected to: Purchases, POS, Stock, Categories and the Audit log.

10.7 Suppliers

Who: manager, stock manager, owner.

Add a supplier. Press **+ New supplier** and fill: **Supplier name** (required), **Business name**, **Phone number**, **Email address**, **Address**, **Status**. The code (SUP-0001…) is automatic.

The list shows the number of purchases, **Total purchased** and **Outstanding** (what you still owe, cancelled purchases excluded). Actions: Edit, History, Activate or Deactivate. **History** lists all purchases from that supplier with totals.

Connected to: the **Supplier** field of a new purchase. Only active suppliers appear there.

10.8 Purchases (buying stock)

Who: manager, stock manager, owner.

The list. Filter by reference, supplier, status and dates. Columns: Reference, Date, Supplier, Total, Paid, Balance, Status.

Step 1: record the purchase. Press **+ New purchase**.

Field	Meaning
Supplier *	an active supplier
Purchase date *	cannot be in the future

Field	Meaning
Notes	optional, up to 255 characters
Products purchased	one line per product: **Product**, **Purchase unit**, **Quantity**, **Unit cost (RWF)**, and the automatic **Line total**
Purchase total	calculated automatically
Amount paid now (RWF)	0, part or all of the total, never more
Payment method	Cash, Mobile Money or Bank
Outstanding supplier balance	calculated: `total − paid now`

The **Unit cost** is the price of one *chosen unit*: if you pick "crate", enter the price of one crate. Press **Save purchase (pending)**. **Stock is NOT increased yet.**

Step 2: confirm when the goods arrive. Open the purchase and press **Confirm & receive stock**. Stock increases (in bottles), the average cost is updated and a Purchase movement is written.

Step 3: pay the supplier. On the purchase page enter an amount (up to the balance), choose the method and press **Record payment to supplier**. You can pay in parts. A **Cash** payment is recorded as cash going out of the open cash session.

Cancel. Type a **reason (required)** and press **Cancel purchase**. A confirmed purchase is reversed (its stock is removed). It is refused if that stock was already sold. In that case use a stock adjustment.

Connected to: Suppliers, Products (stock and average cost), Stock movements and Cash.

10.9 Cash Management

Who: manager, cashier, owner. Deposits and withdrawals: manager and owner only.

You cannot sell until a cash session is open.

Open the session (start of the shift).

1. Count the money in the drawer.
2. Enter it in **Opening cash (RWF)**. The page suggests the cash counted when the last session closed.
3. Press **Open session**. Only one session can be open at a time.

During the shift. The session page shows the **Daily cash reconciliation** and the list of **Cash movements** (In and Out columns). Cash enters the drawer from cash sales, debt payments and deposits. Cash leaves for supplier payments, cash expenses, owner withdrawals and refunds.

- **Cash deposit** (manager): enter the Amount and a Description, for example "Change float added".
- **Owner withdrawal** (manager): enter the Amount and a **Reason** (3 to 255 characters, required). It cannot be more than the expected cash. It is not a business expense and never reduces profit.

Credit sales are not cash received. Mobile Money and bank payments are not counted in the drawer.

Close the session (end of the shift).

1. Count the drawer.
2. Enter **Cash counted in the drawer**.
3. If it differs from the expected amount, a **Note** is required.
4. Press **Close & reconcile**. The system shows the variance. After closing, selling is blocked until a new session is opened.

Connected to: POS, Purchases, Expenses, Debt payments and sale cancellation refunds.

10.10 POS (selling)

Who: manager, cashier, owner. Needs an open cash session and products with stock.

The screen.

- **Left:** a search box (type a name or brand, or scan a barcode and press Enter), category tabs with counts and the product tiles. A tile shows green **N in stock**, orange **N left** (5 or fewer) or red **Out of stock** (cannot be clicked).
- **Right:** the **Current sale** panel.
- **Top:** a green chip showing the open cash session.

How to make a sale.

1. Click a product tile, or scan its barcode and press Enter. Each click adds 1.
2. In the cart use **−** and **+** to change the quantity. Going below 1 removes the line.
3. If the product has several units (crate, pack), use the small unit selector in the line. The price follows the unit.

4. Choose a **Customer**. Leave **Walk-in customer** for a normal cash sale.
5. A manager can enter a **Discount (RWF)**. Cashiers cannot give discounts.
6. Under **Payment** choose **Cash**, **Mobile Money** or **Bank** and enter the amount. **Fill** puts in the exact remaining amount. **+ Add payment (split)** lets one sale be paid in several ways. Mobile Money and Bank also show a **Reference** field for the transaction number.
7. **Paid now** and **On credit (debt)** update by themselves.
8. Press **Complete sale**. You are taken to the receipt.

When the Complete sale button is disabled (a warning explains why):

- the cart is empty
- a line asks for more stock than exists
- the payments are more than the total
- money is left unpaid but no customer is chosen
- the customer has no credit limit, or the new debt would pass the limit (a manager can tick **Authorize over limit**)

Selling on credit. Choose the customer and pay less than the total. The unpaid part becomes the customer's debt.

Connected to: Products (stock goes down), Stock movements, Cash (cash payments), Customers (debt), Sales history and the Audit log.

10.11 Receipt and Sales history

Who: manager, cashier, owner.

Sales history. Filter by receipt number, status and dates. Columns: Receipt, Date, Customer, Sold by, Total, Paid, Credit, Status. Click a receipt to open it.

Receipt page. Buttons: **+ New sale**, **Print / Save as PDF** and **Sales history**.

Cancel a sale (manager and owner). Type a **Reason (required)** and press **Cancel sale**.

- Stock is restored with movement records.
- Any credit is reversed. This is refused if the customer already paid part of that debt.
- The cash part is refunded from the open cash session, so a session must be open.

Sales are never deleted. A cancelled sale stays visible with the status "cancelled".

10.12 Customers and debts

Who: manager, cashier, owner. Changing a customer's status from the list: manager and owner.

The list. Search by name, phone or code. Filter by status. Columns: Code, Customer, Phone, **Credit limit**, **Current debt**, **Available credit** (`limit − debt`), Status. A red **Over limit** badge shows when the debt is above the limit.

Add a customer. Press **+ New customer**.

Field	Meaning
Full name *	required, up to 120 characters
Phone number	7 to 20 characters
Address	up to 255 characters
Credit limit (RWF)	0 means no credit. Only a manager or owner can set it.
Status	active, inactive or blocked (manager and owner only)

The debt itself cannot be typed. It changes only through credit sales and repayments.

Statement / Pay. Open it to see the customer's whole story.

- **Receive a debt payment:** enter the **Amount** (up to the debt), choose the method, add a **Reference** and a **Note** if you want, and press **Record payment**. The oldest debt is paid first. A cash payment goes into the open cash session, so one must be open.
- **Debt statement:** every credit sale and payment with a running **Balance**.
- **Credit sales** and **All purchases** show what the customer bought.

10.13 Expenses

Who: manager and owner.

Add an expense. Press **+ New expense** and fill: **Category** (required), **Amount (RWF)** (required, above 0), **Payment method** (required: Cash, Mobile Money or Bank), **Expense date** (required, not in the future) and **Description** (required, 3 to 255 characters).

A **Cash** expense needs an open cash session and enough money in the drawer. It is then also recorded as cash going out.

The list. Filter by category and dates. It shows the total of the filtered rows. Owner withdrawals are not expenses: record them in Cash Management.

10.14 Stock control

Who: manager, stock manager, owner. The page has two tabs.

Movements. The complete history of every stock change: purchases, sales, cancellations and adjustments. Filter by product, type and dates. Columns: Time, Product, Type, **Change** (+ or −), **Balance after**, Unit cost, Reason / reference, By.

Adjustment records. Only the manual adjustments, with the **Value at cost** and the responsible user.

Make an adjustment. Press **+ Stock adjustment**.

Field	Meaning
Adjustment type *	Damaged, Expired, Missing, Returned or Correction. The first three always decrease stock, Returned always increases it.
Correction direction	shown for Correction only: Decrease or Increase
Product * and Unit	what and in which unit
Quantity *	a whole number of 1 or more
Stock now → after	a preview of the result before you save
Reason *	3 to 255 characters, saved with your name

Stock can never go below zero. The page also warns about expired products still in stock.

10.15 Users (owner only)

Press **+ New user** and fill: **Full name** (required), **Username** (required, 3 to 50 characters: letters, digits, dot, dash, underscore), **Email**, **Phone**, **Role** (required: owner, manager, cashier or stock manager), **Status**, **Password** and **Repeat password** (at least 8 characters with a letter and a digit). When editing, leave both password fields empty to keep the old password.

Rules: you cannot change your own role or status, you cannot deactivate yourself, and there must always be one active owner. An inactive user is logged out on their next click.

10.16 Audit log (owner only)

A read-only history. Filter by **action**, **user** and dates. Columns: Time, User, Action, Record, Details, IP. It shows the latest 300 entries.

10.17 A full example from zero to a sale

1. **Category:** add "Beer".
2. **Product:** "Primus 72cl", base unit Bottle, purchase price 700, selling price 1,000, minimum stock 24, plus a crate unit that contains 20 (leave the price empty to get 20,000).
3. **Supplier:** add "Bralirwa".
4. **Purchase:** supplier Bralirwa, product Primus, unit Crate, quantity 5, unit cost 14,000. The total is 70,000. Pay 30,000 in cash now, so the balance is 40,000. Save it (pending).
5. **Confirm** the purchase. Stock becomes `5 × 20 = 100` bottles. The average cost is `14,000 ÷ 20 = 700` per bottle.
6. **Cash:** open the session with 50,000.
7. **POS:** sell 3 bottles at 1,000 (total 3,000), paid in cash. Stock becomes 97. The expected cash rises by 3,000.
8. **Close** the cash session at the end of the day and count the drawer.

10.18 Daily routine

When	Who	What
Start of the day	cashier or manager	Count the drawer and open the cash session.
During the day	cashier	Sell in the POS. Register new credit customers first. Receive debt payments from the customer's statement.
When goods arrive	stock manager	Open the pending purchase and confirm it. Pay the supplier when due.
When something is lost or broken	stock manager	Make a stock adjustment with a clear reason.

When	Who	What
End of the day	cashier or manager	Count the drawer and close the cash session. Explain any difference in the note.
Weekly	owner	Read the audit log, check low stock and customers over their limit.

11. Function reference

The core toolbox is described in section 5. This section lists the model, service and controller functions. There are 220 functions in total in the project.

11.1 Models (`app/models`): database questions

File	Function	What it does
audit.php	`search_audit_logs($f)`	Audit entries filtered by action, user and dates (latest 300)
audit.php	`get_audit_actions()`	Every action name, for the filter list
categories.php	`get_categories_with_counts()`	All categories with their product count
categories.php	`get_active_categories()`	Active categories for drop-downs
categories.php	`find_category($id)`	One category or null
categories.php	`category_name_exists($name, $excludeId)`	True if the name is already used
categories.php	`create_category`, `update_category`, `set_category_status`	Insert, update, set active or inactive
categories.php	`get_category_product_counts($limit)`	Products per category, biggest first (dashboard)

File	Function	What it does
customers.php	`search_customers($f)`	Customers by text, status and "owes money"
customers.php	`get_active_customers()`	Active customers for the POS
customers.php	`find_customer($id)`	One customer or null
customers.php	`next_customer_code()`	Next CUS-0000 code
customers.php	`create_customer`, `update_customer`, `set_customer_status`	Insert, update (never the debt), set status
customers.php	`get_customer_debts`, `get_customer_payments`, `get_customer_sales`	Credit sales, repayments and latest 100 sales of one customer
dashboard.php	`get_dashboard_counts()`	Counts of products, suppliers, customers, users
expenses.php	`get_expense_categories()`, `find_expense_category($id)`	Active expense categories
expenses.php	`search_expenses($f)`	Expenses by category and dates (latest 500)
products.php	`search_products($f)`	Products by text, category, status and low stock
products.php	`get_active_products()`, `get_all_products()`	Shortcuts for forms
products.php	`get_units_for_products($ids)`	Units of many products at once, grouped by product
products.php	`find_product($id)`, `get_extra_units($id)`	One product, and its units except the base unit
products.php	`get_product_names()`	id, name and code of every product
products.php	`get_expired_products_in_stock()`	Expired products that still have stock

File	Function	What it does
products.php	`next_product_code()`	Next PRD-0000 code
products.php	`create_product($d, $units)`, `update_product($id, $d, $units)`	Save a product and its units in one transaction
products.php	`set_product_status($id, $status)`	Active or inactive
purchases.php	`search_purchases($f)`	Purchases by reference, supplier, status, dates
purchases.php	`find_purchase($id)`, `get_purchase_items($id)`	One purchase with names, and its lines
sales.php	`search_sales($f)`	Sales by receipt, status, dates (latest 300)
sales.php	`find_sale`, `get_sale_items`, `get_sale_payments`	One sale, its lines and its payments
stock.php	`stock_movement_types()`, `stock_adjustment_types()`	The allowed type names
stock.php	`search_stock_movements($f)`, `search_stock_adjustments($f)`	Movement history and adjustment records
suppliers.php	`get_suppliers_with_totals()`	Suppliers with purchase count, total and outstanding
suppliers.php	`get_supplier_list()`, `get_active_suppliers()`	Lists for filters and the purchase form
suppliers.php	`find_supplier`, `next_supplier_code`, `create_supplier`, `update_supplier`, `set_supplier_status`	Basic supplier work
suppliers.php	`get_supplier_summary($id)`	Totals over non-cancelled purchases
users.php	`find_user_by_username($login)`	Login lookup by username or email
users.php	`touch_last_login($id)`	Saves the last login time

File	Function	What it does
users.php	`get_all_users()`, `find_user($id)`, `get_roles()`	Users and roles
users.php	`username_exists`, `email_exists`, `count_active_owners`	Uniqueness and owner checks
users.php	`create_user($d, $hash)`, `update_user($id, $d, $hash)`, `set_user_status`	Save users

11.2 Services (`app/services`): business rules

File	Function	What it does
audit.php	`audit_log($userId, $action, $type, $id, $details)`	Writes one audit entry. Never stops the app if it fails.
cash.php	`get_open_cash_session()`	The open session or null
cash.php	`find_cash_session($id)`, `get_cash_history($limit)`	One session, latest sessions
cash.php	`open_cash_session($opening, $userId)`	Opens the till
cash.php	`record_cash_transaction(...)`	Writes one cash movement
cash.php	`get_cash_summary($id)`	Totals and expected cash
cash.php	`get_cash_transactions($id)`	Movements of a session
cash.php	`close_cash_session($id, $counted, $notes, $userId)`	Closes the till, calculates variance
cash.php	`deposit_cash`, `withdraw_cash`	Money in, owner withdrawal
cash.php	`assert_cash_session_open($id)`	Stops with an error if the session is not open
debts.php	`pay_customer_debt(...)`	Applies a repayment to the oldest debts first

File	Function	What it does
expenses.php	`next_expense_code()`, `create_expense($d, $userId)`	Records an expense (and the cash movement)
purchases.php	`next_purchase_reference()`	Next PUR-YYYYMMDD-0000
purchases.php	`lock_purchase($id)`, `lock_product_row($id)`	Read a row and lock it for the transaction
purchases.php	`record_purchase_movement(...)`	Writes a purchase stock movement
purchases.php	`create_purchase($h, $rows, $userId)`	Saves a pending purchase (retries a duplicate reference)
purchases.php	`confirm_purchase($id, $userId)`	Goods received: stock up, average cost updated
purchases.php	`cancel_purchase($id, $reason, $userId)`	Cancels, reversing stock if needed
purchases.php	`pay_purchase(...)`, `record_purchase_payment(...)`	Pays the supplier
sales.php	`next_receipt_number()`	Next REC-YYYYMMDD-0000
sales.php	`create_sale($in, $userId, $role)`	The whole sale in one transaction
sales.php	`cancel_sale($id, $reason, $userId)`	Cancels a sale and reverses stock, debt and cash
stock.php	`apply_stock_adjustment($d, $userId)`	Damaged, expired, missing, returned or correction

11.3 Controllers (`app/controllers`): one function per page

GET functions show a page. POST functions handle a form and then redirect.

Function	Address	Allowed
`dashboard_index`	GET /, /dashboard	any logged-in user

Function	Address	Allowed
`auth_show_login`, `auth_login`	GET, POST /login	public
`auth_logout`	POST /logout	logged in
`category_index`, `category_store`, `category_update`, `category_toggle`	/categories, /categories/store, /update, /toggle	manager, stock manager
`product_index`, `product_create`, `product_store`, `product_edit`, `product_update`, `product_toggle`	/products, /create, /store, /edit, /update, /toggle	manager, stock manager
`supplier_index`, `supplier_create`, `supplier_store`, `supplier_edit`, `supplier_update`, `supplier_toggle`, `supplier_history`	/suppliers and sub-addresses	manager, stock manager
`purchase_index`, `purchase_create`, `purchase_store`, `purchase_show`, `purchase_confirm`, `purchase_cancel`, `purchase_pay`	/purchases and sub-addresses	manager, stock manager
`pos_index`, `pos_checkout`	GET /pos, POST /pos/checkout	manager, cashier
`sale_index`, `sale_receipt`	/sales, /sales/receipt	manager, cashier
`sale_cancel`	POST /sales/cancel	manager
`cash_index`, `cash_session`, `cash_open`, `cash_close`	/cash and sub-addresses	manager, cashier
`cash_deposit`, `cash_withdraw`	POST /cash/deposit, /cash/withdraw	manager
`customer_index`, `customer_create`, `customer_store`, `customer_edit`, `customer_update`, `customer_statement`	/customers and sub-addresses	manager, cashier

Function	Address	Allowed
`customer_toggle`	POST /customers/toggle	manager
`debt_pay`	POST /debts/pay	manager, cashier
`expense_index`, `expense_create`, `expense_store`	/expenses and sub-addresses	manager
`stock_index`, `stock_adjustments`, `stock_adjust`, `stock_store`	/stock and sub-addresses	manager, stock manager
`user_index`, `user_create`, `user_store`, `user_edit`, `user_update`, `user_toggle`, `user_audit`	/users and sub-addresses	owner

The owner is allowed on every row. Helper functions inside controller files (not routed): `product_save`, `product_collect_input`, `product_db_error_message`, `product_show_form`, `product_unit_names`, `product_base_units`, `purchase_collect_input`, `purchase_show_form`, `customer_save`, `customer_show_form`, `compare_ledger_rows`, `supplier_save`, `supplier_show_form`, `user_save`, `user_show_form`, `expense_show_form`, `stock_show_form`.

11.4 JavaScript files (`public/assets/js`)

File	Used on	Job
login.js	login page	show or hide password, field checks, remember username, forgot-password note
pos.js	POS	tiles, categories, search and barcode, cart, payments, totals, checkout request
products.js	product form	add and remove extra unit rows, update the base-unit label
purchases.js	new purchase	add and remove lines, fill unit and cost from the product, calculate totals
adjust.js	stock adjustment	list units of the chosen product, show stock before and after, show the direction field for corrections

11.5 Views and shared parts (`app/views`)

Folder or file	Job
layouts/main.php	The frame of every page after login: sidebar, page header, flash messages, content
layouts/auth.php	The frame of the login page
partials/sidebar.php	The fixed menu, filtered by role, with the Log out button
partials/pagehead.php	Page title, date, user name, and the cash-session chip on the POS
partials/icons.php	The SVG icon set used by the menu and buttons
partials/product_unit_row.php	One extra unit row of the product form
partials/purchase_item_row.php	One line of the purchase form
one folder per module	`index.php` for lists, `form.php` or `create.php` for forms, `show.php` , `receipt.php` , `statement.php` , `history.php` for details

12. Security

Protection	How it is done
Passwords	Stored only as `password_hash` (bcrypt). Checked with `password_verify`.
SQL injection	Every query uses `?` placeholders through the `db_*` functions.
Script injection (XSS)	Every printed value passes through `e()`.
Forged forms (CSRF)	Every POST form has a secret token that the router checks.
Double submit	The POS sends a one-time token with each sale.
Session fixation	A new session id is made at login and logout.
Inactive accounts	The account is re-read from the database on every request.
Roles	Checked on every route, and again for sensitive rules inside services.
Folders	`.htaccess` sends every request to `public/`, so `app/`, `config/` and `database/` cannot be opened in the browser. Directory listing is off.
Audit trail	Sensitive actions are written to `audit_log` with the user and IP address.
Concurrent changes	`SELECT ... FOR UPDATE` locks product, customer and purchase rows inside transactions.
Prices	The POS price is always read from the database, never from the browser.

12.1 Before going live

1. Set `'debug' => false` in `config/app.php`.
2. Change every sample password, or delete the sample accounts.
3. Use a MySQL account that is not `root`, with a password, and put it in `config/database.php`.

4. Use HTTPS if the system is reachable from outside the bar.
5. Do not leave phpMyAdmin open to the internet.
6. Set up regular database backups (see section 13).

13. Maintenance

- **Backups.** Export the database often (phpMyAdmin "Export", or `mysqldump`). Keep copies on another device. The `database/backup` folder is there to keep dumps, but it is on the same machine, so also copy it elsewhere.
- **Updating the code.** Copy the new files over the old ones. Keep your own `config/database.php` and `config/app.php`.
- **Adding a user.** Use the Users page (owner). Never share one account between people, because the audit log records who did what.
- **Resetting a password.** The owner edits the user and types a new password. It is logged as `password_reset`.
- **Fixing wrong stock.** Do not edit numbers in the database. Use a stock adjustment with a reason, so the history stays honest.
- **A wrong sale.** Cancel it (manager) and make a new one.

14. Extending the system

14.1 Add a new page (example: a Reports page)

1. **Query** (if needed): add `get_sales_totals()` in `app/models/sales.php`.
2. **Controller:** create `app/controllers/report.php` with `function report_index(): void { render('reports/index', ['title' => 'Reports', 'active' => 'reports', 'totals' => get_sales_totals()]); }`
3. **Load it:** add `require APP_PATH . '/controllers/report.php';` to `app/bootstrap.php`.
4. **Route:** add `['GET', '/reports', 'report_index', ['roles' => ['manager']]],` to `routes/web.php`.
5. **View:** create `app/views/reports/index.php` (plain HTML with `<?= e($value) ?>`).
6. **Menu:** in `app/views/partials/sidebar.php` change the Reports line so its "built" flag is `true`.

14.2 Add a form that saves data

1. Put `<?= csrf_field() ?>` inside the `<form method="post">`.
2. Add a POST route and a controller function. In it, read values with `post_text()` and `post_int()`, check them, and collect messages in an `$errors` array.
3. If there are no errors, call a model function to save, call `audit_log()` for important actions, set `flash('success', '...')` and `redirect(...)`.
4. If there are errors, call `render()` on the form again with what the user typed, so nothing is lost.
5. If the save changes several tables, put it in a service function and wrap it in `db_begin()` and `db_commit()` with the `try / catch` rollback.

14.3 Rules to keep the code easy to read

- One function does one job.
- Controllers read and check input. Models ask questions. Services protect money and stock.
- Always use `?` placeholders, never put values inside the SQL text.
- Always use `e()` when printing.
- Never change stock or money outside a service that uses a transaction.
- Put a short comment above every function that says what it does.

15. Troubleshooting

Problem	Likely cause and fix
Blank white page	An error is hidden because debug is off. Set `'debug' => true` in `config/app.php`, reload and read the message.
"404 Page not found" on every page except the login	`mod_rewrite` is off or `AllowOverride` is not `All`. Enable the rewrite module in Apache and restart.
Error about the database connection	Check `config/database.php` and that MySQL is running.

Problem	Likely cause and fix
"Table ... doesn't exist"	`schema.sql` was not imported. Import it, then `seed.sql`.
Login says "Invalid username or password"	Wrong password, or the account is inactive. The sample password is `Admin@123`.
"Session expired or invalid form token" (419)	The page stayed open too long or the session ended. Go back, reload the page and try again.
"403 You do not have permission"	The user's role cannot open that page. Check section 6.
"Open a cash session before selling"	Open a session in Cash Management first.
Complete sale button stays grey	A warning in the cart explains why: empty cart, too much stock requested, unpaid amount without a customer, credit limit passed, or payments above the total.
"Not enough stock"	The product has fewer bottles than requested. Receive a purchase first, or sell less.
"Cannot cancel" a purchase	Some of that stock was already sold. Use a stock adjustment.
A sale cannot be cancelled	The customer already repaid part of its credit, or no cash session is open for the refund.
Login background video does not play	The browser blocks autoplay or the file is missing from `public/assets/video/`. The login still works without it.
Styles look broken	Clear the browser cache (Ctrl+F5) and check that `public/assets/css/design.css` exists.

16. Test checklist

Use this after installing or after any change.

1. Log in as `admin`. A wrong password shows the error message. Logging out works.
2. As the owner, create a user for each role and log in as each one. Check that menus and pages match section 6.

3. Create a category, a product with a crate unit, and a supplier.
4. Create a purchase. Check that stock did not change. Confirm it. Check that stock and average cost changed. Pay the supplier in two parts.
5. Open a cash session. Make a cash sale in the POS. Open the receipt and print it.
6. Make a credit sale to a customer with a limit. Try a sale above the limit as a cashier (it must be refused) and as a manager with "Authorize over limit".
7. Receive a debt payment on the customer's statement. Check that the oldest debt is paid first.
8. Record a cash expense and an owner withdrawal. Check the expected cash.
9. Make a stock adjustment (damaged). Try one larger than the stock (it must be refused).
10. Cancel the first sale. Check the stock, the debt and the cash refund.
11. Close the cash session with a counted amount that differs from expected. A note must be required.
12. Open the audit log and check that all of the above appears.

17. Known limits

- **Reports** are not built yet (the menu shows "soon"). The data needed for profit is already stored (section 9.7).
- **Password reset** is done by the owner. There is no email reset.
- **"Remember me"** only remembers the username in the browser. It does not keep you logged in longer.
- **Long lists** show the latest records only: 500 stock movements, 500 purchases, 500 expenses, 300 sales, 300 audit entries.
- **One till.** Only one cash session can be open at a time.
- **One currency.** All amounts are in RWF and prices are whole numbers in the forms.
- **No delete.** Records are cancelled or set to inactive, never deleted, by design.
- **Testing note.** The conversion to functions was checked with a static analysis script and the login page was tested in a browser. The PHP and database behaviour should be confirmed with the checklist in section 16 on your own server.

18. Glossary

Term	Meaning
Base unit	The smallest unit stock is counted in (bottle or can).
Conversion factor	How many base units are inside a selling unit (a crate of 24 has factor 24).
Average cost (weighted)	The cost per bottle, balanced across purchases made at different prices.
Pending purchase	A purchase that is recorded but whose goods have not arrived. It does not change stock.
Confirm	Marks the goods as received. Stock goes up.
Cash session	One shift of the till, from the opening count to the closing count.
Expected cash	What the drawer should hold: opening + cash in − cash out.
Variance	Counted cash minus expected cash.
Credit sale	A sale where part or all of the total is paid later by a registered customer.
Credit limit	The most debt a customer is allowed to owe. 0 means no credit.
Split payment	One sale paid with several methods, for example cash and Mobile Money.
Stock movement	One row in the history of stock: what changed, by how much, and why.
Adjustment	A manual stock change (damaged, expired, missing, returned, correction) with a reason.
Audit log	The record of who did which sensitive action and when.
Flash message	A message shown once on the next page, such as "Saved".

Term	Meaning
CSRF token	A secret value in every form that proves the form came from this website.
Transaction	A group of database changes saved all together or not at all.
Controller, model, service, view	The four kinds of code: page handler, database question, business rule, HTML.